

DEVs PROJECT



METODOLOGIA DE LA PROGRAMACIÓN

Contenidos de hoy

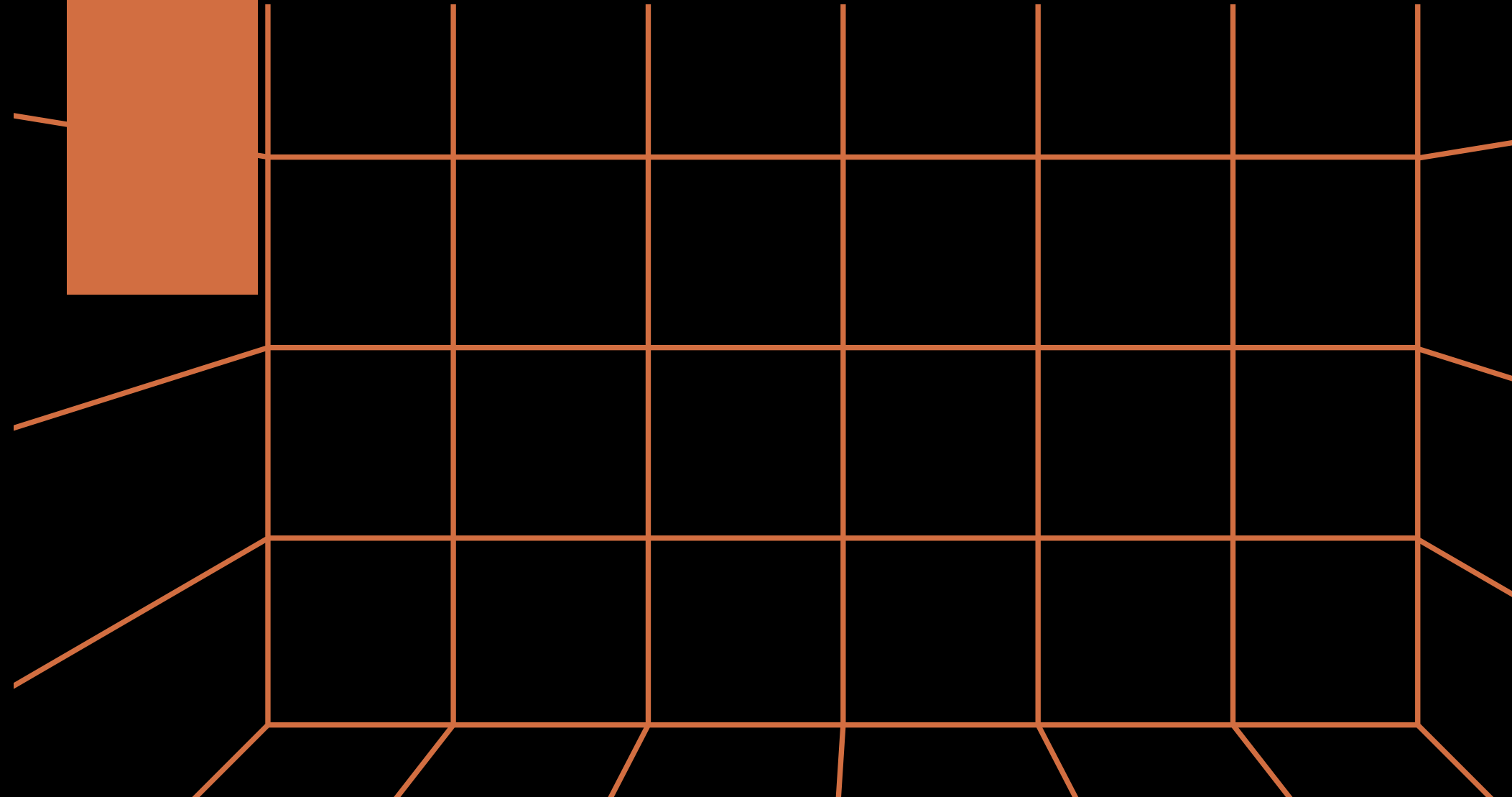


- MANEJO DE GIT
- EJERCICIOS CON PYTHON Y GIT



GIT

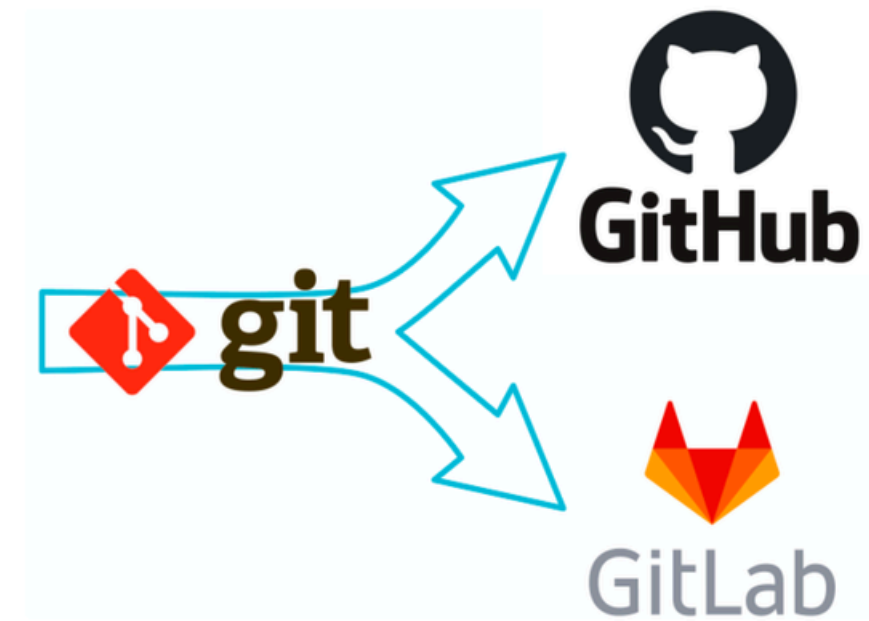
Dominando el Control de Versiones:
Git y GitHub



GIT | DEFINICIÓN

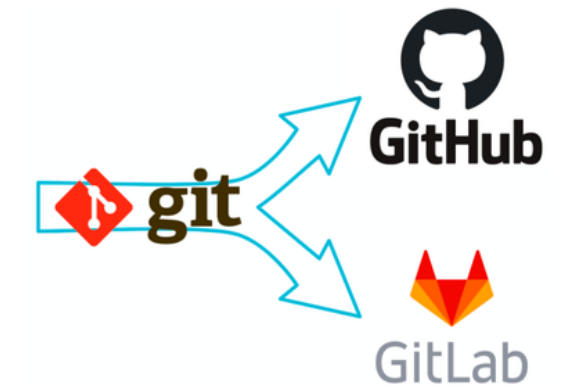
Es un sistema que ayuda a organizar el código, el historial y su evolución, funciona como una **máquina del tiempo** que permite navegar a diferentes versiones del proyecto y si queremos agregar una funcionalidad nueva nos permite crear una rama (*branch*) para dejar intacta la versión estable y crear un ambiente de trabajo en el cual podemos trabajar en una nueva funcionalidad sin afectar la versión original. Permite:

- Manejar distintas versiones del proyecto.
- Guardar el historial o guardar todas las versiones de los archivos del proyecto.
- Trabajar simultáneamente sobre un mismo proyecto.



Git vs GitHub vs Gitlab

Git es el motor; GitHub es el garaje en la nube.



Git (Local)

Sistema de control de versiones distribuido. Funciona localmente en tu máquina. Gestiona el historial y los 'puntos de guardado'.



- **Gaming:** Es como 'Guardar Partida' en tu consola.



- **Office:** Es como el 'Control de Cambios' de Word.



- **Video:** Es el archivo de video en tu disco duro.

GitHub (Nube)

Servicio de alojamiento en la nube. Facilita la colaboración, copias de seguridad y distribución.



- **Gaming:** Es como Steam (plataforma para compartir).



- **Office:** Es como Google Drive (colaboración en tiempo real).

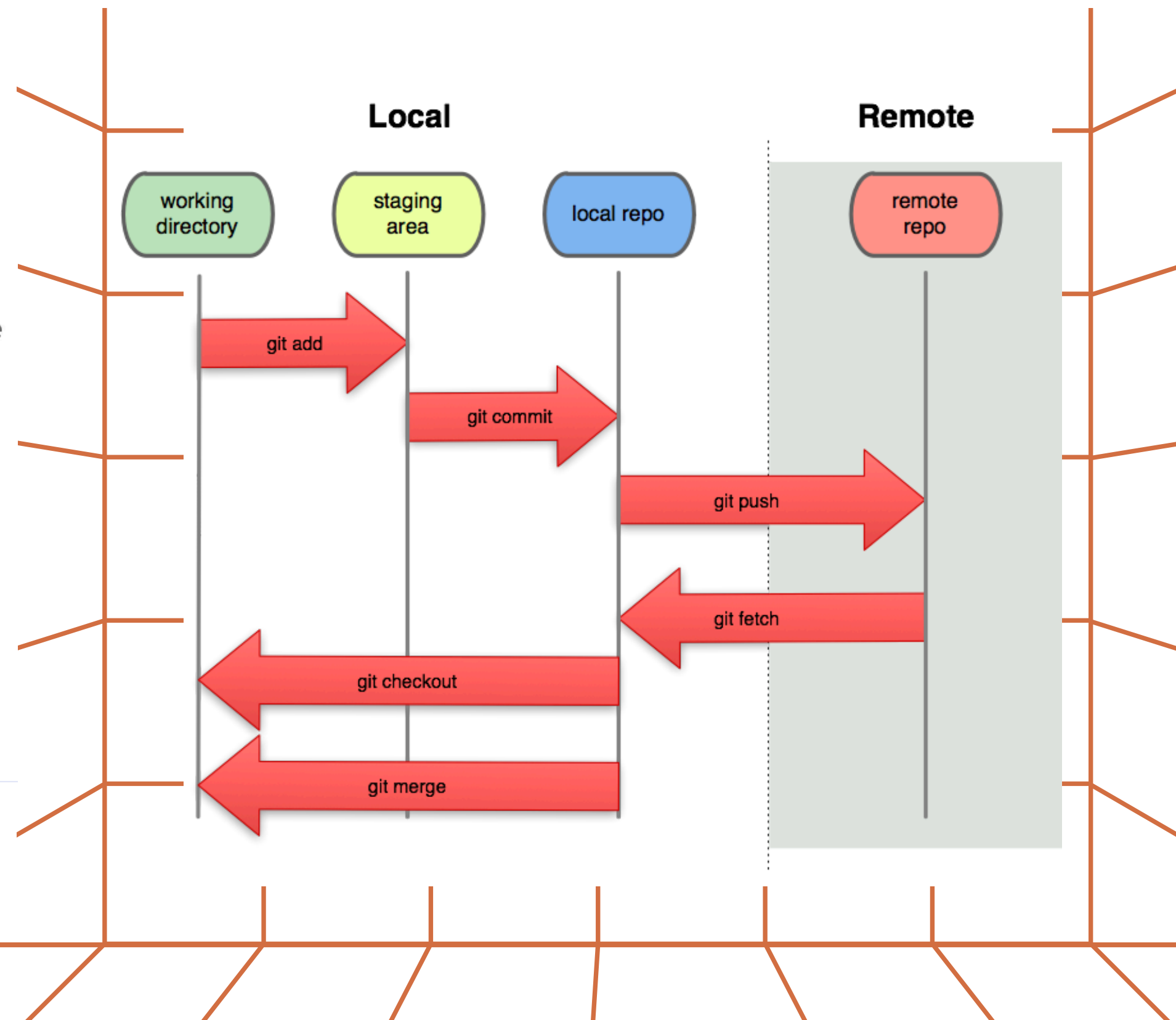


- **Video:** Es YouTube (donde el mundo ve tu video).

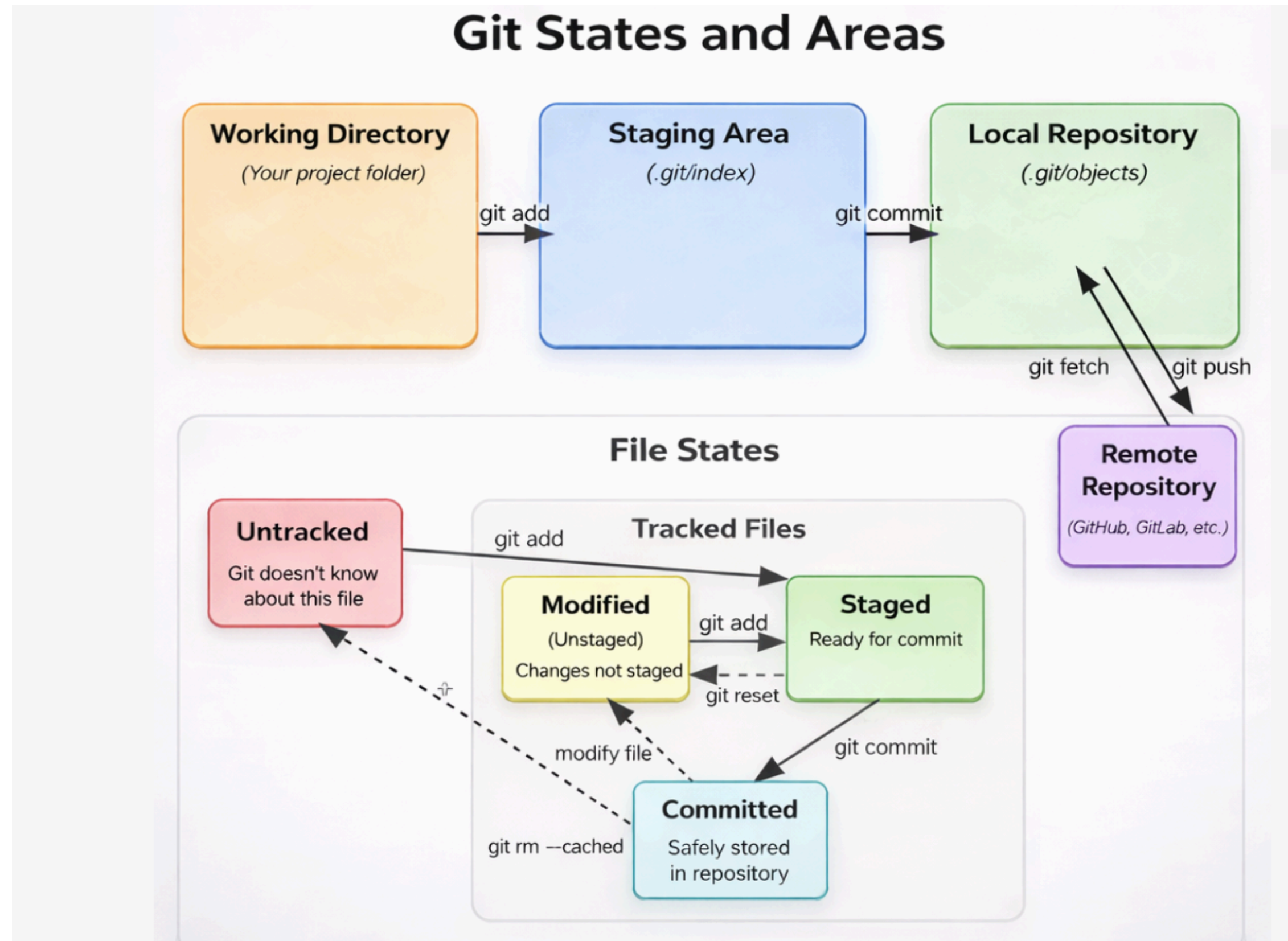
GIT | FUNCIÓN

¿Como funciona?

Git almacena instantáneas de un mini sistema de archivos. Cada vez que confirmamos un cambio, Git toma una "foto" del aspecto del proyecto en ese momento y crea una referencia a esa instantánea. Si un archivo no cambió Git sólo crea un enlace a la imagen anterior idéntica que tiene almacenada.



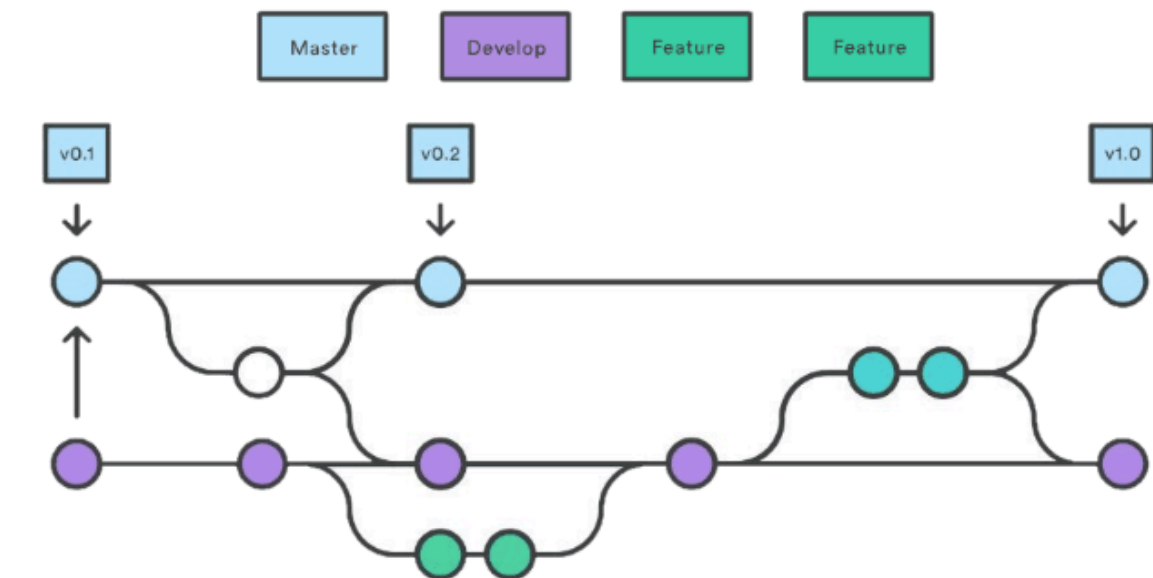
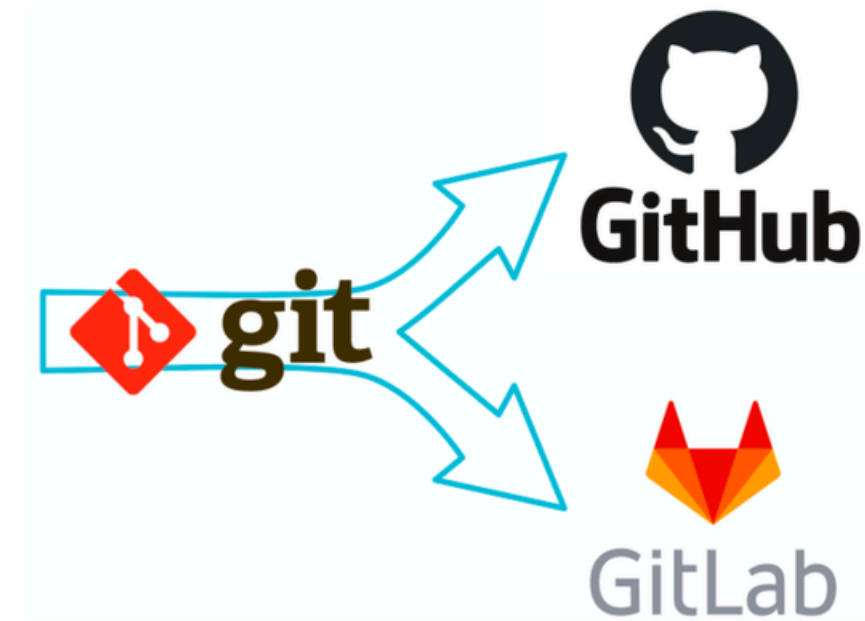
El Flujo de Trabajo (The Git Workflow)





TERMINOLOGÍA

- **Repositorio:** es la carpeta principal donde se encuentran almacenados los archivos que componen el proyecto. El directorio contiene *metadatos* gestionados por **Git**, de manera que el proyecto es configurado como un repositorio local.
- **Commit:** un *commit* es el estado de un proyecto en un determinado momento de la historia del mismo, imaginemos esto como punto por punto cada uno de los cambios que van pasando. Depende de nosotros determinar cuántos y cuales archivos incluirá cada **commit**.
- **Rama (branch):** una rama es una línea alterna del tiempo en la historia de nuestro repositorio. Funciona para crear *features*, arreglar *bugs*, experimentar, sin afectar la versión estable o principal del proyecto. La rama principal por defecto es **master**.
- **Pull Request:** en proyectos con un equipo de trabajo, cada persona puede trabajar en una rama distinta, pero llegado el momento puede pasar que dicha rama se tenga que unir a la rama principal. Para eso se crea un *pull request* donde comunicamos el código que incluye los cambios, es revisado, comentado y aprobado para darle *merge*. En el contexto de GIT, *merge* significa unir dos trabajos, en este caso la rama *branch* con la rama *master*.



INSTALACIÓN

<http://git-scm.com/install/>

Install

Latest version: 2.52.0 (Release Notes)

Windows macOS Linux Build from Source

Click here to download the latest (2.52.0) x64 version of Git for Windows. This is the most recent **maintained build**. It was released **about 2 months ago**, on 2025-11-17.

Other Git for Windows downloads

Standalone Installer
Git for Windows/x64 Setup.

Git for Windows/ARM64 Setup.

Portable ("thumbdrive edition")
Git for Windows/x64 Portable.

Git for Windows/ARM64 Portable.

Using winget tool

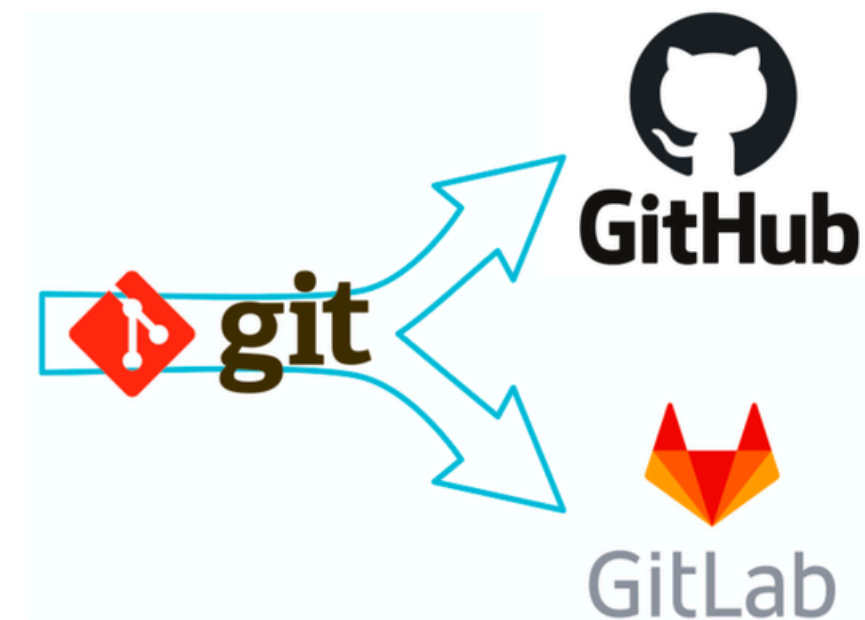
Install **winget tool** if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

The current source code release is version **2.52.0**. If you want the newer version, you can build it from **the source code**.



git



Instalación Multiplataforma: Preparando el Entorno

Una guía paso a paso para Windows, macOS y Linux.



Windows

1. Descargar el instalador oficial.
2. Ejecutar asistente: Siguiente → Siguiente → Finalizar.
3. Verificar en terminal:

```
git --version
```



macOS

Opción A (Recomendada):

```
git --version
```

(Suele instalarse automáticamente con Xcode Command Line Tools)

Opción B: Usar instalador oficial.



Linux

Ubuntu/Debian:

```
sudo apt-get install git
```

Fedora:

```
sudo dnf install git
```

Pro Tip: Si ves una salida como 'git version 2.x.x', la instalación fue exitosa.

Identidad y Seguridad: Configuración Inicial

Identidad (Obligatorio)

Git necesita firmar cada cambio con tu nombre y correo.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

Nombre

Tu Nombre

Correo

tu@email.com

verificar
git config --global --list

Seguridad SSH (Recomendado)

Evita escribir tu contraseña en cada interacción.
Configuración en 3 pasos:

1. Generar Clave

```
ssh-keygen -t rsa -b 4096 -C "tu@email.com"
```

2. Activar Agente

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_rsa
```

3. Vincular en GitHub

Copiar contenido de 'id_rsa.pub' →
GitHub Settings → SSH Keys.



```
ssh -T git@github.com
```

Comandos Esenciales: Iniciar y Capturar

Crear y Obtener

`git init [nombre]`

Crea un nuevo repositorio local desde cero.

`git clone [url]`

Descarga un proyecto existente y todo su historial.

Efectuar Cambios

`git status`

Lista archivos nuevos o modificados.

`git add [archivo]`

Prepara el archivo para la versión (Staging).

`git diff`

Muestra diferencias antes de preparar.

`git commit -m "[mensaje]`

Registra la instantánea permanentemente.

`git reset [archivo]`

Saca un archivo del área de preparación.

Conectando con Repositorios Remotos

Concepto

Crimson Pro: Un 'remoto' es el enlace entre tu carpeta local y GitHub.

Añadir Conexión

```
git remote add origin  
https://github.com/USUARIO/REPO.git
```

'origin' es el alias estándar para el servidor principal.

Verificar

```
git remote -v
```

Muestra las URLs de lectura (fetch) y escritura (push).

Solución de Problemas

Error: fatal: remote origin already exists

Solución: El nombre 'origin' ya está en uso. Usa 'git remote set-url' para actualizarlo o 'git remote rename' para cambiar el nombre.

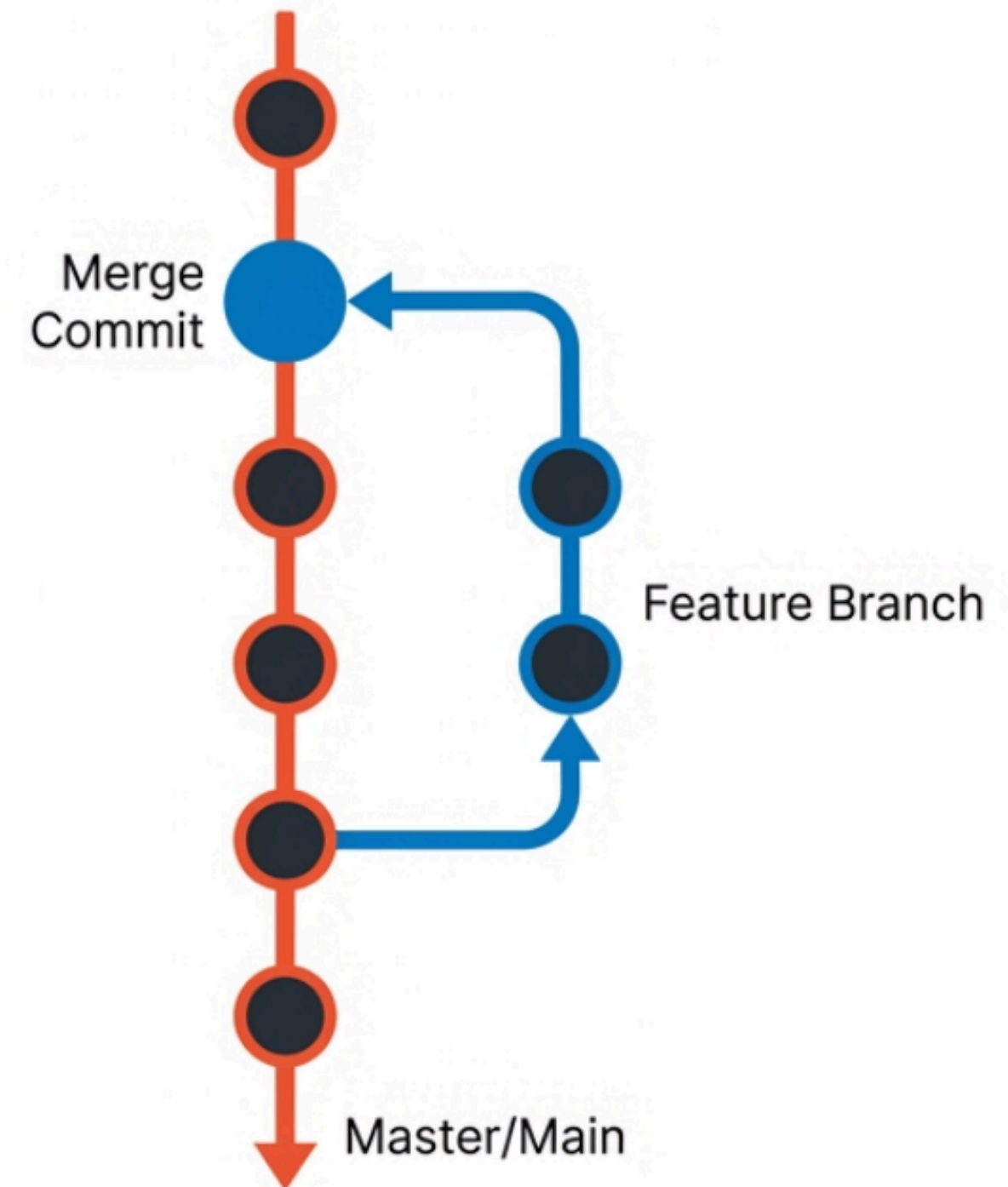
Comandos Esenciales: Ramas y Fusión

Gestión de Ramas (Branching)

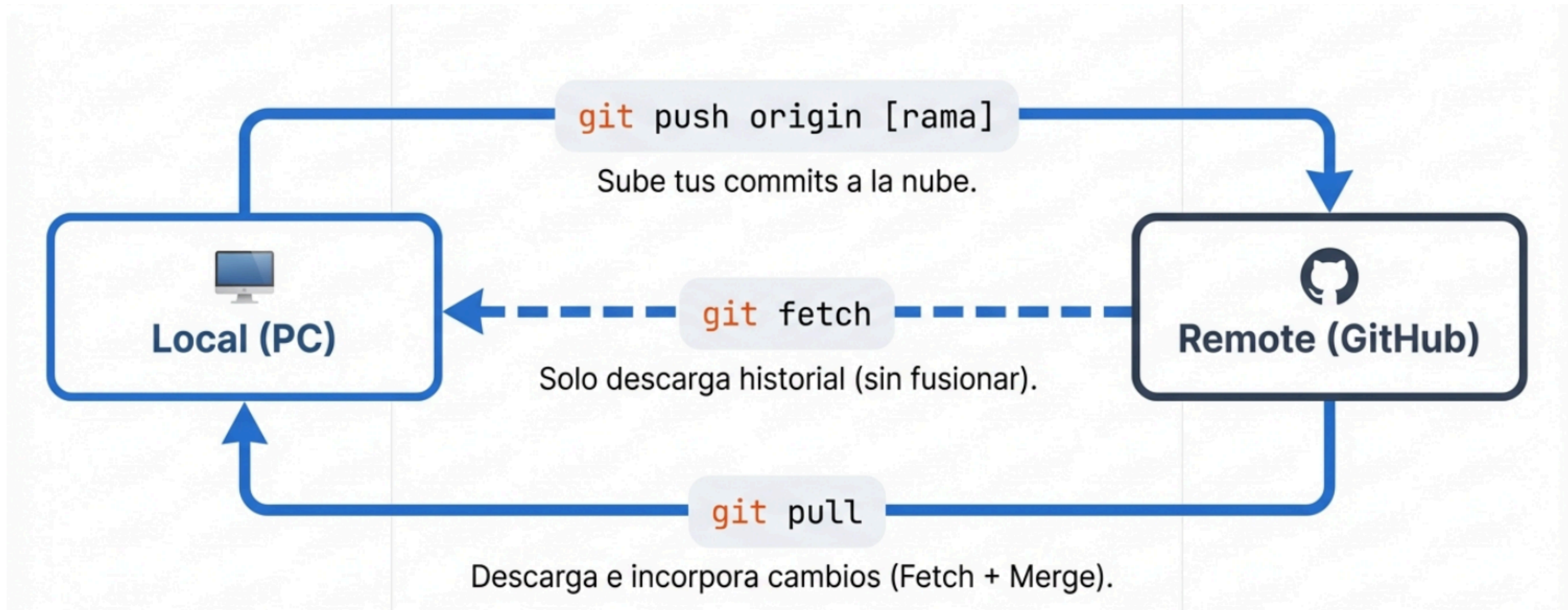
- `git branch` Lista las ramas locales.
- `git branch [nombre]` Crea una nueva rama.
- `git checkout [nombre]` Cambia a la rama seleccionada.
- `git branch -d [nombre]` Borra una rama.

Combinar Cambios (Merging)

- `git merge [rama]` Fusiona el historial de la rama especificada con la actual.



Sincronización: Push & Pull



Flujo típico: git pull (actualizar) → Trabajar → git commit → git push (publicar).

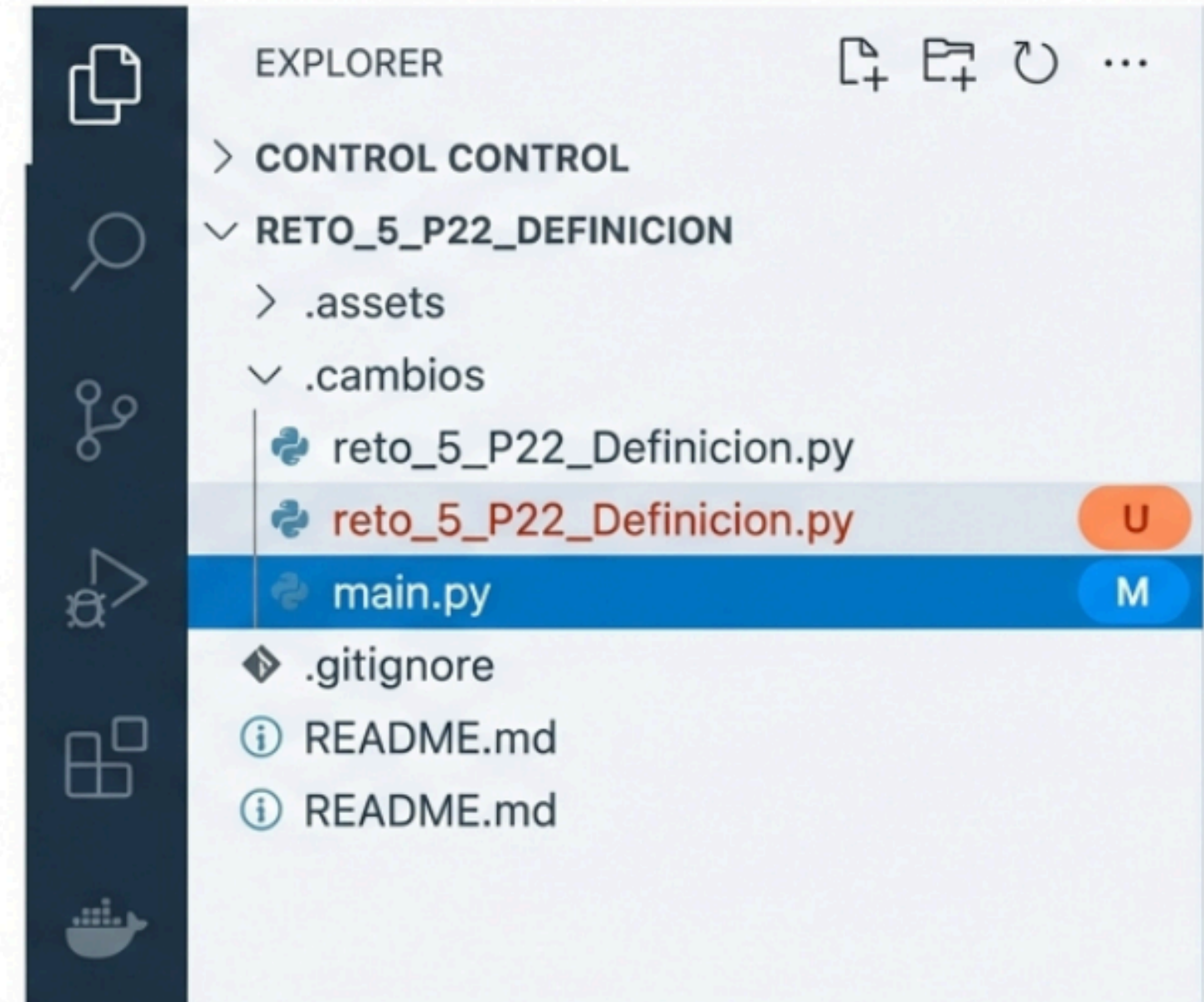
COMANDOS QUE USARÁN SIEMPRE

- Ver estado: **git status**
 - Preparar cambios: **git add**
 - Guardar cambios: **git commit**
 - Subir cambios: **git push**
 - Actualizar proyecto: **git pull**
 - Copiar proyecto: **git clone**
-

Integración Visual: Git en VSCode (Parte 1)

Inicialización y Estado

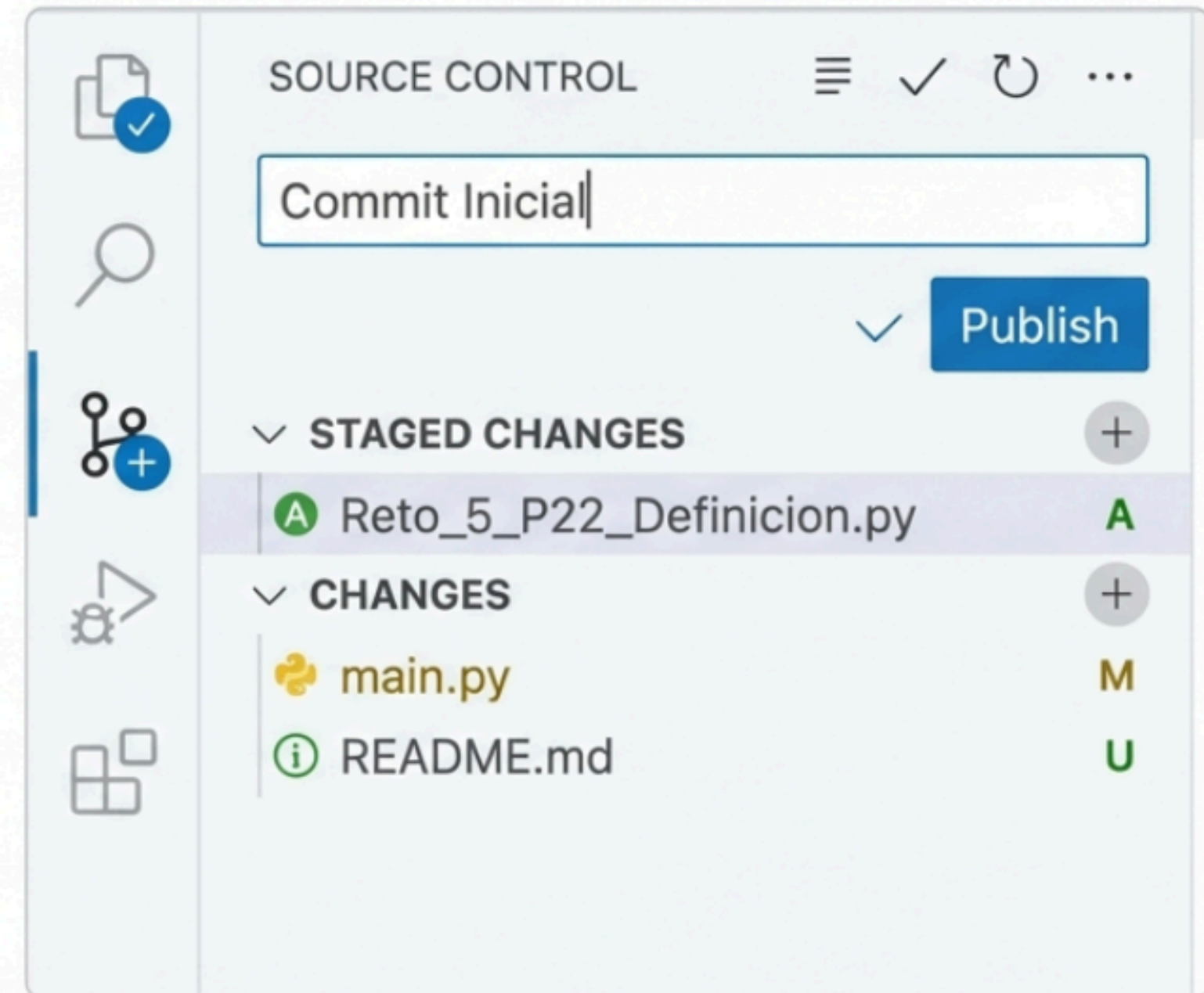
1. Inicializar: Abre la carpeta en VSCode.
Usa la terminal integrada (Ctrl + ñ) o el botón de la barra lateral.
2. Decodificando la Interfaz:
 - **U (Untracked)**: Archivo nuevo, sin seguimiento.
 - **M (Modified)**: Archivo con cambios no guardados.
 - **A (Added)**: Archivo listo en Staging.



Integración Visual: Git en VSCode (Parte 2)

El Ciclo Visual: Stage, Commit y Publish

1. **Stage (+)**: Clic en el signo '+' para mover cambios a 'Stored Changes'.
2. **Commit (✓)**: Escribe tu mensaje en la caja superior y confirma con Ctrl+Enter.
3. **Publish**: Usa el botón 'Publicar en GitHub' para sincronizar.



El Arte del Commit: Reglas de Oro

Tu historial cuenta una historia. Hazla legible.

Regla 1: Imperativo

Usa verbos como órdenes:
Add, Fix, Change.



Good: ``Fix login bug``



Bad: ``Fixed login bug``

Truco: Completa la frase "Si aplico este commit, va a..."

Regla 2: Sin Puntuación Final

Es un título, no una oración.
Evita puntos finales y puntos suspensivos.



Example: ``Add search feature`` (No ``.``)

Regla 3: Concisión

Máximo 50 caracteres en el título. Detalles extra van en la descripción.

TÍTULO (≤ 50 chars)

DESCRIPCIÓN
(Detalles extra)

Commits Semánticos

`<tipo>(scope): <descripción>`

Prefijos Comunes

feat: Nueva característica

fix: Corrección de error

docs: Cambios en documentación

Prefijos Comunes

style: Formato (espacios, ;)

refactor: Reescritura sin cambios funcionales

test: Añadir o corregir pruebas

Ejemplos Reales

`feat(backend): add filter for cars`

`fix(css): remove wrong color in navbar`

Resumen y Siguietes Pasos

Tu Caja de Herramientas

- ✓ Instalar Git y configurar claves SSH.
- ✓ Definir identidad (`user.name`).
- ✓ Dominar el ciclo: `init` → `add` → `commit` → `push`.
- ✓ Aplicar Commits Semánticos e Imperativos.

Recursos y Filosofía

- Recursos: Documentación oficial de GitHub, Git SCM, Cheatsheets.

*“**Git** no es solo una herramienta para guardar código; es la **máquina del tiempo** que gestiona la evolución y la historia de tus ideas.”*

¿PREGUNTAS?

Si no hay preguntas pasemos con un ejercicio con git :)



EJERCICIO EN CLASE

Resolver una consigna de programación aplicando buenas prácticas de código (Clean Code, funciones con una sola responsabilidad, uso correcto de métodos) y gestionar el proyecto con Git, desde la inicialización del repositorio hasta la subida del código a GitHub, siguiendo el flujo visto en clase.

La empresa necesita un sistema simple para gestionar datos de sus empleados. Para ello, se deberán utilizar tres listas paralelas, donde cada posición representa a un mismo empleado.

- Lista de nombres de empleados
- Lista de edades
- Lista de salarios

El sistema deberá permitir:

1. Agregar un empleado

- Antes de agregar, se debe validar que el empleado no exista previamente en la lista de nombres.
- Si existe, se debe informar al usuario y no agregarlo.
- Si no existe, se agregan los datos en las tres listas respetando la posición.
-

2. Buscar un empleado

- Se debe validar si el empleado existe antes de mostrar su información.
- Si no existe, se debe informar correctamente.
-

3. Comparar salarios

- El sistema deberá filtrar y mostrar cuál de los empleados tiene el salario mayor.
- La comparación debe realizarse recorriendo las listas y respetando la relación entre ellas.